

Forecasting U.S. Immigration Rates with Regression and Machine Learning

Gurpreet Khamba

2025-12-16

Contents

Introduction	1
Library	1
Datasets	2
Python Data Scrape uses Playwright	2
Data Cleaning using R	3
Reading in the Datasets	4
Filtering Datasets	5
Models	6
Baseline Models	6
Lagged Outcome Regression Models	6
Random Forest	7
Evaluation	8
Conclusion	13

Introduction

In this small project I will forecast annual U.S immigration counts using historical immigration data and economic indicators. A LOCF (last observation carried forward) and a linear time trend baseline are compared against a lag-based regression and a Random Forest model. Results show that lagged immigration counts are the dominant predictor.

Library

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.1      v tibble    3.2.1
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.0.4
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(randomForest)
```

```
## randomForest 4.7-1.2
## Type rfNews() to see new features/changes/bug fixes.
##
## Attaching package: 'randomForest'
##
## The following object is masked from 'package:dplyr':
##
##   combine
##
## The following object is masked from 'package:ggplot2':
##
##   margin
```

```
library(gt)
library(readxl)
```

Datasets

Immigration data was scraped from Office of Homeland of Security Statistics website, using Python and Playwright and then compiled into a data set using R. GDP and Unemployment Rates were available online.

Python Data Scrape uses Playwright

```
from playwright.sync_api import sync_playwright
import os

DOWNLOAD_DIR = "downloads"
os.makedirs(DOWNLOAD_DIR, exist_ok=True)

#After 2014, zip files are used for storage
SKIP_YEARS = [str(y) for y in range(2015, 2023)]

with sync_playwright() as p:
```

```

browser = p.chromium.launch(headless=False)
context = browser.new_context(accept_downloads=True)
page = context.new_page()

page.goto("https://ohss.dhs.gov/topics/immigration/lawful-permanent-residents/profiles")
page.select_option(
    'select[name="table_cob_length"]',
    value='-1'
)

page.locator("#table_cob tbody tr").first.wait_for()

links = page.locator("a[href]").all()

for link in links:
    link_text = link.inner_text().strip()
    href = link.get_attribute("href")

    if not href:
        continue

    if not href.lower().endswith((".xlsx", ".xls")):
        continue

    if any(year in link_text for year in SKIP_YEARS):
        print("Skipping:", link_text)
        continue

    with page.expect_download() as d:
        link.click()

    download = d.value
    ext = download.suggested_filename.split(".")[1]
    safe_name = link_text.replace(" ", "_").replace("/", "_")
    filename = f"{safe_name}.{ext}"

    download.save_as(os.path.join(DOWNLOAD_DIR, filename))
    print("Downloaded:", filename)

browser.close()

```

Data Cleaning using R

```

files <- list.files("C:/Users/death/OneDrive/Documents/Forecasting Immigration Project/OHSS_Immigration",
length(files)

tidy_file <- function(file_path){
  data <- read_excel(file_path, sheet = 1, col_names = FALSE)

  colnames(data) <- c("Characteristic", "Total", "Female", "Male", "Unknown")

```

```

data_year <- str_extract(data[1,1], "\\d+")
data_country <- str_extract(data[4,1], "(?<=:)(.*)")
data <- data|>
  mutate(Year = data_year,
         Country = data_country)|>
  filter(!is.na(Characteristic))|>
  mutate(
    Category = case_when(
      Characteristic %in% c("Total") ~ "Total",
      Characteristic %in% c("New arrivals") ~ "New arrivals",
      Characteristic %in% c("Adjustments of status") ~ "Adjustments of status",
      Characteristic %in% c("Age") ~ "Age",
      Characteristic %in% c("Marital status") ~ "Marital status",
      Characteristic %in% c("Occupation") ~ "Occupation",

      Characteristic %in% c("Broad class of admission",
                           "Major class of admission") ~
        "Broad class of admission",

      Characteristic %in% c("Leading states/territories of residence",
                           "Leading states of residence",
                           "Top 20 states of permanent residence") ~
        "Leading states/territories of residence",

      TRUE ~ NA_character_
    )
  )|>
  fill(Category)|>
  filter(!is.na(Total)&!is.na(Category))|>
  mutate(across(2:5, ~ ifelse(. %in% c("-", "D"), 0, .)))|>
  mutate(across(2:6, as.numeric))|>
  pivot_longer(cols = 2:5,
               names_to = "Sex",
               values_to = "Value")

data <- data[,c(3,2,4,1,5,6)]
}

all_data <- map_dfr(files, tidy_file)

View(all_data)
#write.csv(all_data, "OHSS Immigration Data.csv", row.names = FALSE)

immigration_total <- read.csv("OHSS Immigration Data.csv")

```

Reading in the Datasets

```

immigration_total <- read.csv("OHSS Immigration Data.csv")
gdp_total <- read.csv(skip = 4,
                     check.names = FALSE, "Global_GDPs.csv")
unempl_total <- read.csv(skip = 4,

```

```
check.names = FALSE, "Unemployment_Rates.csv")
```

Filtering Datasets

Because the immigration data spans 2003–2023, the GDP and unemployment data were restricted to the same period to ensure temporal alignment.

```
gdp_total[c(2:47,69,70)] <- NULL
gdp_total <- gdp_total|>
  rename(Country = "Country Name")|>
  pivot_longer(cols = (2:22),
               names_to = "Year",
               values_to = "GDP",
  )|>
  mutate(Year = as.numeric(Year))

unempl_total[c(2:47,69,70)] <- NULL
unempl_total <- unempl_total|>
  rename(Country = "Country Name")|>
  pivot_longer(cols = (2:22),
               names_to = "Year",
               values_to = "Unemployment_Rate",
  )|>
  mutate(Year = as.numeric(Year))

immigration_numbers <- immigration_total|>
  filter(Characteristic == "Total",
         Sex == "Total"
  )|>
  select(Country, Year, Value)|>
  rename(Immigrants = Value)

combined_total <- unempl_total|>
  left_join(gdp_total, by = c("Country","Year"))
  )|>
  left_join(immigration_numbers, by = c("Country","Year"))
  )|>
  filter(!is.na(Immigrants))

##Allows for more intuitive graphs.
combined_total <- combined_total |>
  mutate(log_immigrants = log1p(Immigrants))

combined_total <- combined_total |>
  filter(Immigrants > 0)|>
  mutate(log_immigrants = log(Immigrants))
```

Models

Baseline Models

Last Observation Carried Forward (LOCF) used to show how well we could forecast using only the previous year's immigration.

Linear Trend used to test whether a simple time trend plus country differences can capture overall growth or decline.

```
#LOCF baseline
locf_baseline <- combined_total |>
  group_by(Country) |>
  mutate(
    pred_locf = lag(log_immigrants, 1)
  ) |>
  filter(!is.na(pred_locf)) |>
  ungroup()

rmse_locf <- sqrt(mean((locf_baseline[["log_immigrants"]] - locf_baseline$pred_locf)^2))

#Linear Time Trend baseline
base_year <- 2014
train_df <- combined_total |> filter(Year <= base_year)
test_df <- combined_total |> filter(Year > base_year)

##Only countries that exist in training
###identical(test_df,train_df)
###anti_join(test_df,train_df, by = "Country")
test_base_lm_df <- test_df |>
  filter(Country %in% train_df$Country)

##Fit trend model on training
trend_model <- lm(log_immigrants ~ Country + Year, data = train_df)
test_base_lm_df <- test_base_lm_df |>
  mutate(pred_trend = predict(trend_model, newdata = test_base_lm_df))

rmse <- function(y, yhat) sqrt(mean((y - yhat)^2))

rmse_trend <- rmse(test_base_lm_df$log_immigrants, test_base_lm_df$pred_trend)
```

Lagged Outcome Regression Models

Immigration levels tend to be persistent over time, hence using previous year's value and economic indicators, we predict the future. One model uses GDP and the Unemployment Rate, while a simpler model include GDP only. Additional predictors such as country indicators were explored but they did not improve performance and in some cases worsened RMSE, so they were not retained.

```
#adjust train and test data frames for lag
train_lm_df <- train_df |>
  group_by(Country) |>
  mutate(lag_immigrants = lag(log_immigrants, n = 1)) |>
  ungroup()
```

```

test_lm_df <- test_df |>
  group_by(Country) |>
  mutate(lag_immigrants = lag(log_immigrants, n = 1)) |>
  ungroup()

lm_model <- lm(log_immigrants ~ lag_immigrants + GDP + Unemployment_Rate,
              data = train_lm_df)

test_lm_df <- test_lm_df |>
  mutate(pred_lm = predict(lm_model, newdata = test_lm_df)) |>
  filter(!is.na(pred_lm))

#without unemployment rate
lm_model_2 <- lm(log_immigrants ~ lag_immigrants + GDP,
                data = train_lm_df)

test_lm_2_df <- test_lm_df |>
  mutate(pred_lm = predict(lm_model_2, newdata = test_lm_df)) |>
  filter(!is.na(pred_lm))

rmse_lm_1 <- rmse(test_lm_df$log_immigrants, test_lm_df$pred_lm)
rmse_lm_2 <- rmse(test_lm_2_df$log_immigrants, test_lm_2_df$pred_lm)

```

Random Forest

A Random Forest model is included as a machine learning approach. Unlike AR and linear models, it does not assume linear relationships or explicit temporal structure, instead learning predictive patterns directly from the data.

```

features <- c(
  "lag_immigrants",
  "GDP",
  "Year"
)

train_rf_df <- na.omit(train_lm_df)
test_rf_df <- na.omit(test_lm_df)

rf_model <- randomForest(
  log_immigrants ~ lag_immigrants + GDP + Year,
  data = train_rf_df,
  ntree = 500,
  importance = TRUE
)

test_rf_df <- test_rf_df |>
  mutate(pred_rf = predict(rf_model, newdata = test_rf_df))

rmse_rf <- rmse(test_rf_df$log_immigrants, test_rf_df$pred_rf)
#0.4288735 with test_lm_2_df in test_rf_df

```

Model Performance: RMSE Comparison

Model	RMSE	Difference from LOCF (%)
LOCF	0.293	0.0
Linear Trend	0.579	97.7
Lagged Outcome (GDP+Unemployment)	0.284	-3.0
Lagged Outcome (GDP only)	0.284	-3.3
Random Forest	0.407	38.9

Evaluation

The Lagged Outcome Regression model with GDP achieves an RMSE of 0.2835, significantly better than the Random Forest model with an RMSE of 0.421. The model achieves a 3.3% lower RMSE than the LOCF baseline and a 51% lower RMSE than the linear trend model.

LOCF performed very well because immigration flows are very persistent over time. Linear time trends assume smooth change over time and global events, such as Covid, greatly impacted it. It overfit a trend in the training that did not continue. Lagged outcome regression performed well as it was able to adapt to short-term deviations without enforcing a global trend. Random Forest has flexibility but substantial cross-country variation in immigration levels may have impacted its forecasting accuracy.

Only a few economic indicators were included. Amongst which, unemployment rates negatively impacted the model. Policies may have a greater impact on legal immigration.

Illegal Alien population forecasts can be less impacted by policies and thus more accurate, but accurate data is not available on their populations.

```
#Table
rmse_table <- data.frame(
  Model = c("LOCF", "Linear Trend", "Lagged Outcome (GDP+Unemployment)", "Lagged Outcome (GDP only)", "Random Forest"),
  RMSE = c(rmse_locf, rmse_trend, rmse_lm_1, rmse_lm_2, rmse_rf)
)

rmse_table[["Difference from LOCF (%)"]] <- (rmse_table$RMSE - rmse_locf)/rmse_locf *100
rmse_table$RMSE <- round(rmse_table$RMSE, 3)
rmse_table[["Difference from LOCF (%)"]] <- round(rmse_table[["Difference from LOCF (%)"]], 1)

rmse_table |>
  gt() |>
  tab_header(
    title = "Model Performance: RMSE Comparison"
  )
```

```
#Visual Graph - combining all model predictions
all_rmse_df <- locf_baseline |>
  filter(Year %in% test_rf_df$Year) |>
  filter(Country %in% test_rf_df$Country) |>
  left_join(test_rf_df |> select(Country, Year, pred_lm, pred_rf),
    by = c("Country", "Year")) |>
  left_join(test_base_lm_df |> select(Country, Year, pred_trend),
    by = c("Country", "Year"))
```



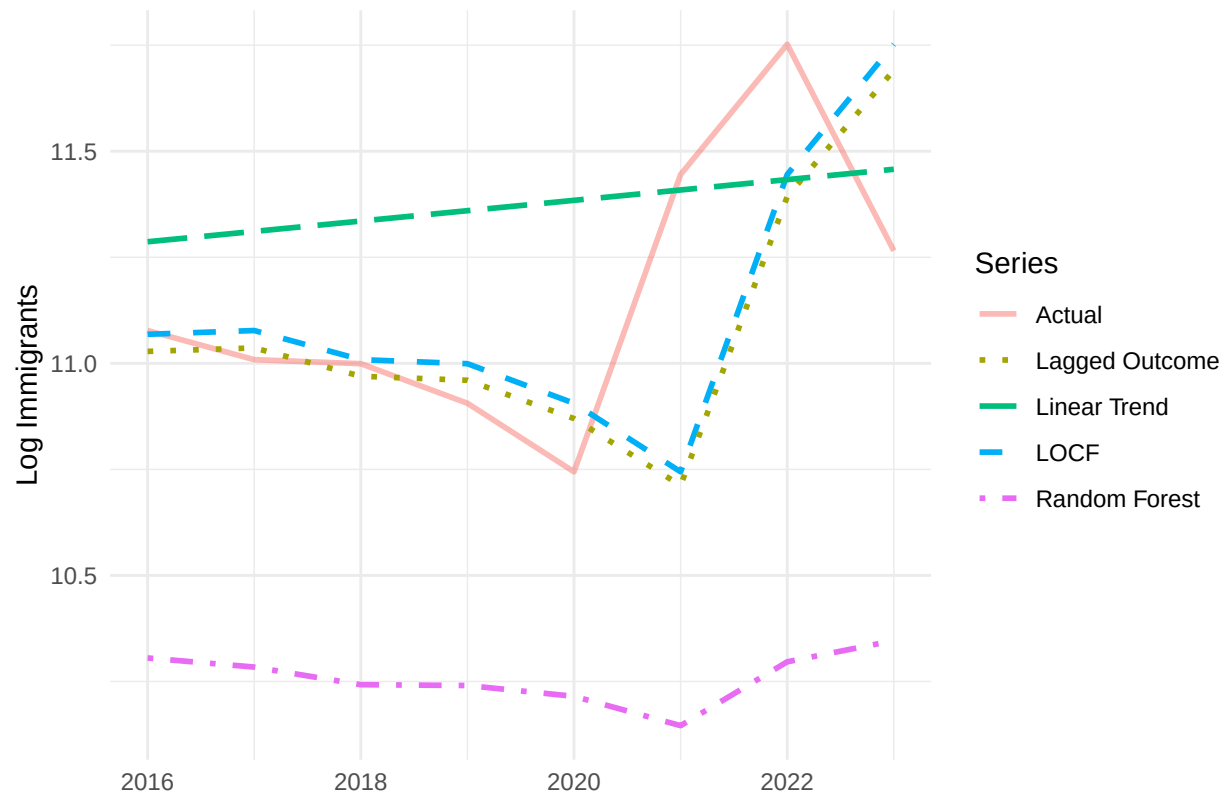
```

#visual graph function
rmse_graph <- function(country){
  all_rmse_df |>
    filter(Country == country) |>
    ggplot(aes(x = Year)) +
    geom_line(aes(y = log_immigrants, color = "Actual"),
              linetype = "solid", linewidth = 1, alpha = 0.5) +
    geom_line(aes(y = pred_locf, color = "LOCF"),
              linetype = "dashed", linewidth = 1) +
    geom_line(aes(y = pred_lm, color = "Lagged Outcome"),
              linetype = "dotted", linewidth = 1) +
    geom_line(aes(y = pred_rf, color = "Random Forest"),
              linetype = "dotdash", linewidth = 1) +
    geom_line(aes(y = pred_trend, color = "Linear Trend"),
              linetype = "longdash", linewidth = 1) +
    theme_minimal() +
    theme(plot.title = element_text(hjust = 0.5)) +
    theme(plot.title = element_text(size = 14, face = "bold")) +
    labs(title = paste("Observed vs Predicted Immigration -", country),
         y = "Log Immigrants",
         x = NULL,
         color = "Series")
}

#Countries that have high immigration amounts
rmse_graph("India")

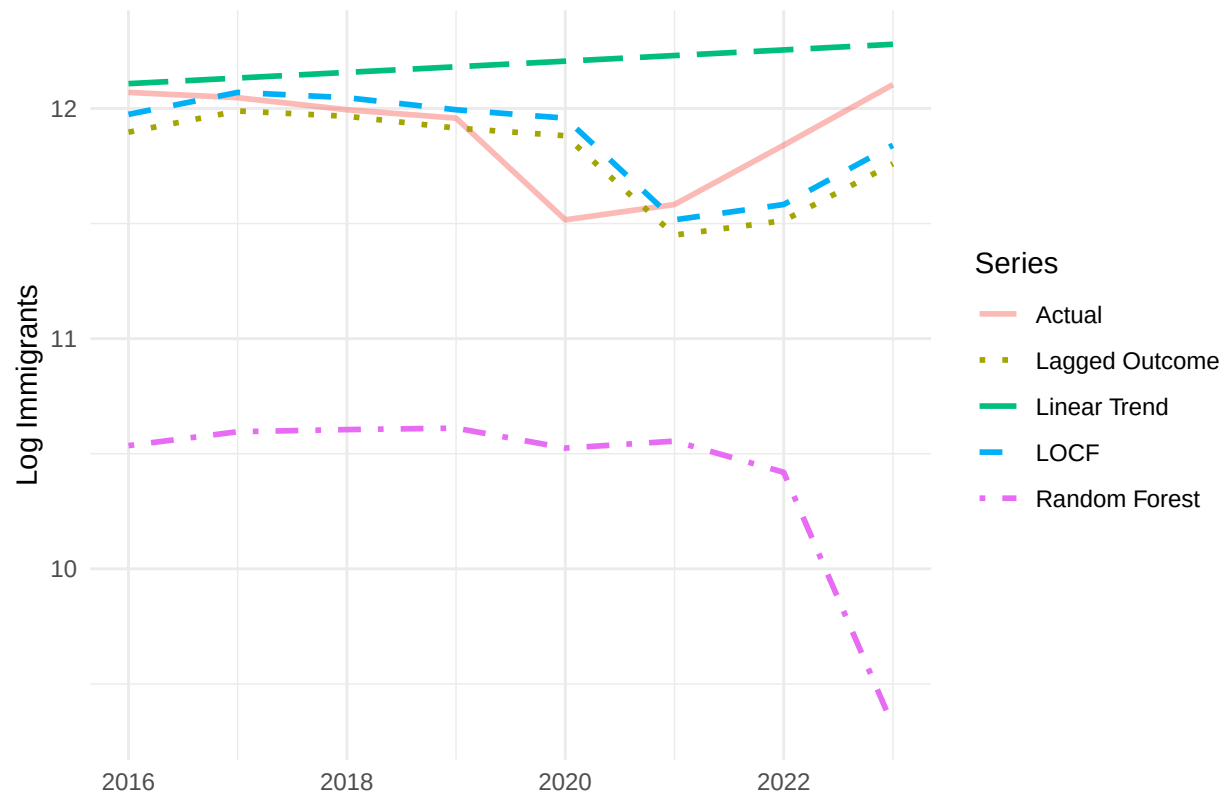
```

Observed vs Predicted Immigration — India



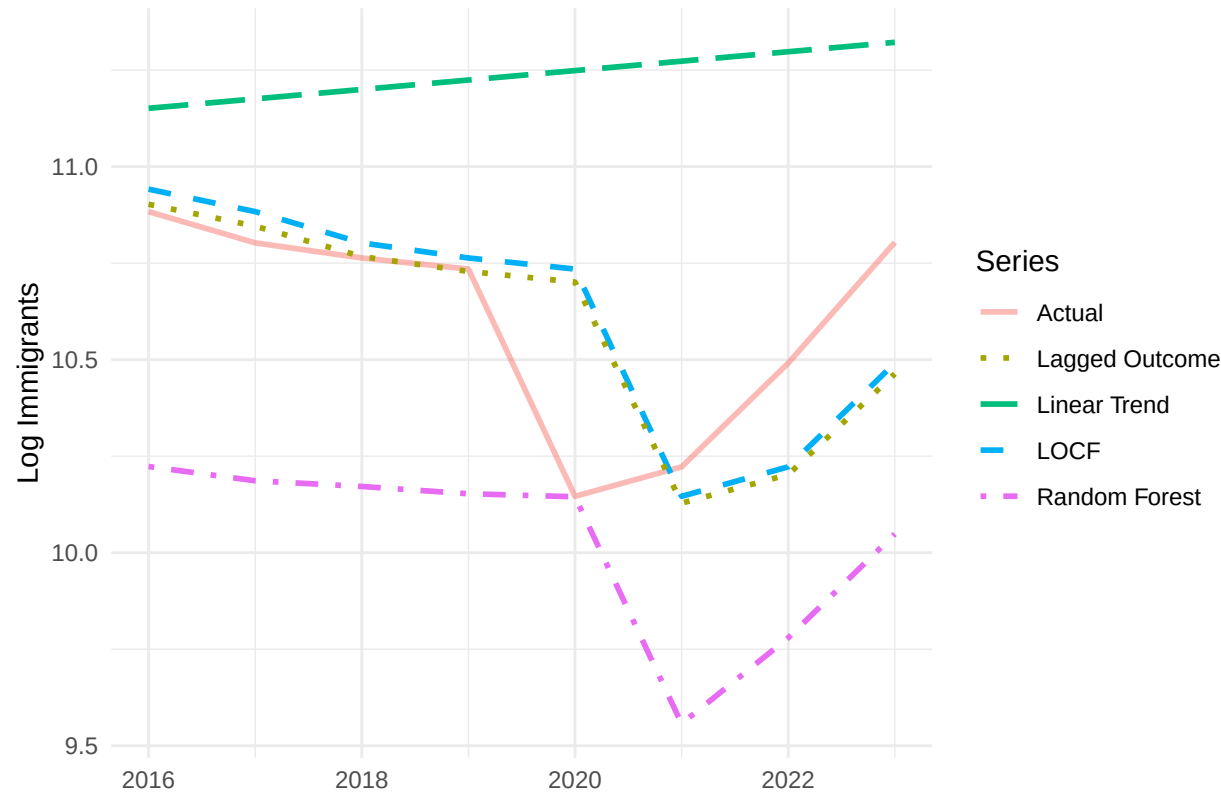
```
rmse_graph("Mexico")
```

Observed vs Predicted Immigration — Mexico



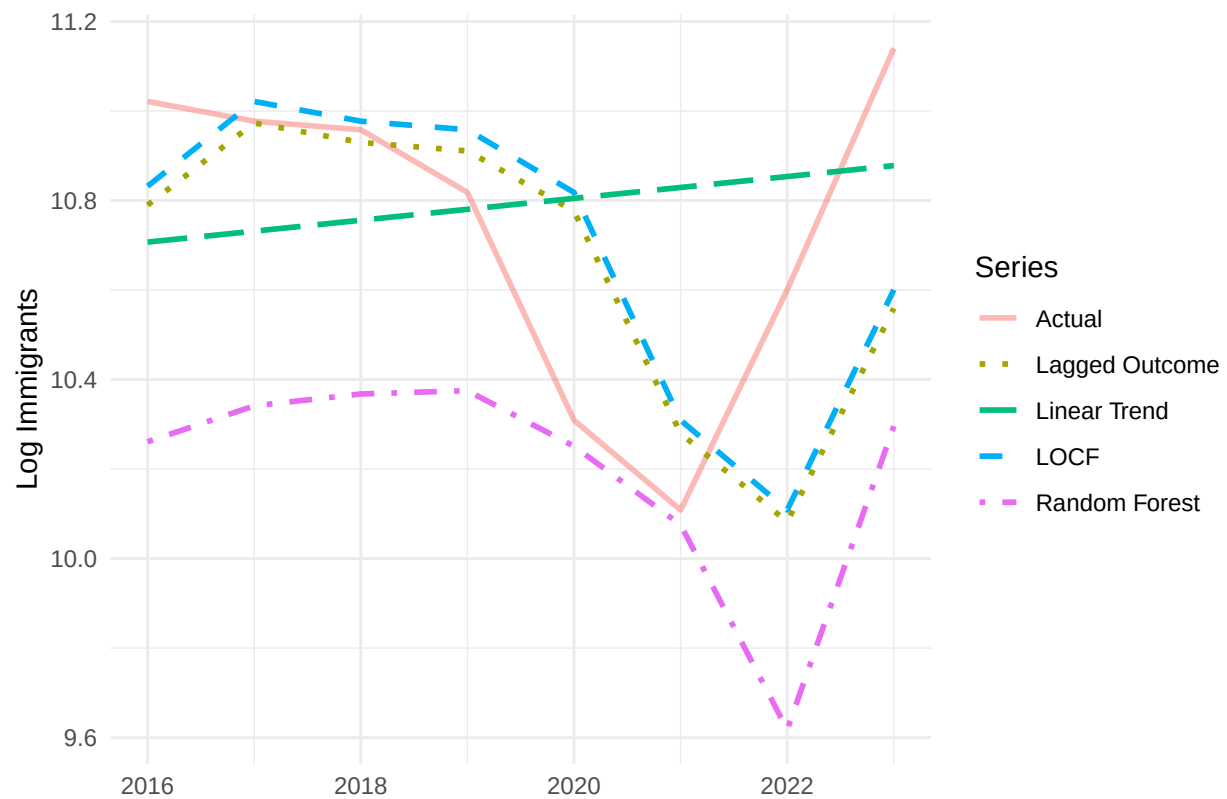
```
rmse_graph("Philippines")
```

Observed vs Predicted Immigration — Philippines



```
rmse_graph("Dominican Republic")
```

Observed vs Predicted Immigration — Dominican Republic



Conclusion

Overall, lagged outcome regression performed best, highlighting the string persistence in immigration flows over time. Although the Random Forest model was a flexible machine learning approach, it did not outperform the simpler lagged regression. This suggests that past immigration levels are highly informative for short-term forecasting and incorporating additional economic variables could further improve predictions.